

Git Status erklären

Betrachten Sie die folgende Ausgabe von `git status` in einer lokalen Workingcopy (Arbeitskopie):

```
pm-lecture % git status
On branch b03

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   CONTRIBUTING.md
    modified:   homework/b03.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    foo.java

no changes added to commit (use "git add" and/or "git commit -a")
```

Erklären Sie die Ausgabe:

In der ersten Zeile gibt man den Befehl `git status` ein. Der Rest dannach ist die Ausgabe nach dem Befehl.

Die Informationen die man bekommt sind, dass man sich auf dem *branch b03* befindet und gibt eine Warnung, dass man noch die Veränderungen die man bei *CONTRIBUTING.md* und *homework/b03.md* noch nicht gestaged hat, also noch nicht hinzugefügt hat. Dies erkennt man an dem *modified:* welches davor bei beiden steht. Zusätzlich hat man bei den untracked files noch *foo.java*. Dies ist keine direkte Veränderung von einer Datei die schon drin war, sondern eine die man komplett neu hinzufügen will. Es steht auch häufig das man die *modified* und *untracked* Sachen adden soll mit `git add <file>`. Am Ende steht noch etwas von `git commit -a`. Würde man diesen Befehl ausführen, würde es automatisch die modified Dateien adden und committen aber nicht die untracked file *foo.java*.

Geben Sie eine Befehlssequenz an, mit der Sie nur die Änderungen in *foo.java* committen können:

```
git status
git add foo.java
git status
git commit -m "foo.java wurde hinzugefügt"
```

1) Öffnen Sie eine Konsole und beantworten Sie mit Hilfe der Befehle `git checkout`, `git log` und `git show` sowie `git diff` folgende Fragen:

Was passierte an tag 01?

Benutzter Befehl:

`git log --oneline` zeigt die Commit-Historie kurz und kompakt.

`git show 4aa8014` zeigt, was in dem Commit mit der ID 4aa8014 geändert wurde.

Antwort:

In einer düsteren und mysteriösen Welt wagte sich ein furchtloser Held namens Markus in einen gefährlichen Dungeon. Es wurde erzählt, dass in diesem Dungeon Monster lauerten, die die Kerkerebenen terrorisierten. Markus war jedoch fest entschlossen, die Monster zu besiegen und das Amulet zu finden.

+

+Mit seinem Schwert in der Hand und einer leichten Rüstung bekleidet stieg Markus tapfer in den Dungeon ein. Schon bald stieß er auf die ersten Monster - grässliche, rattenähnliche Wesen.

Wann hat der Held zum ersten Mal 4 experience Punkte?

Benutzter Befehl:

`git log -p -- stats.md` zeigt, wann stats.md geändert wurde und was genau in der Datei geändert wurde (von allen commits).

Antwort:

Am Tag 01.3 hat der Held zum ersten Mal 4 experience Punkte.

Wann hat der Held zum ersten Mal 10 hunger Punkte?

Benutzter Befehl:

`git log -p -- stats.md` zeigt, wann stats.md geändert wurde und was genau in der Datei geändert wurde (von allen commits).

Antwort:

Am Tag 02 hat der Held zum ersten Mal 10 hunger Punkte.

Wie viele Heiltränke hat der Held insgesamt in seinem Rucksack gehabt?

Benutzter Befehl:

`git log -p -- rucksack.md` zeigt, wann rucksack.md geändert wurde und was genau in der Datei geändert wurde (von allen commits).

Antwort:

Am Tag 03.14 hat der Held einen Heiltrank bekommen.

Am Tag 03.17 hat der Held den Heiltrank verbraucht.

Am Tag 04.4 hat der Held einen Heiltrank bekommen.

Also hatte der Held insgesamt 2 Heiltränke in seinem Rucksack gehabt.

Was hat der Held im Shop gekauft? Und wie viel Gold hat er dafür bezahlt?

Benutzter Befehl:

`git log -p -- rucksack.md`

zeigt, wann rucksack.md geändert wurde und was genau in der Datei geändert wurde (von allen commits).

`git log -p -- shopkeeper.md`

zeigt, wann shopkeeper.md geändert wurde und was genau in der Datei geändert wurde (von allen commits).

Antwort:

Der Held hat am Tag 03.9 ein Brot für 5 Gold gekauft.

Der Held hat am Tag 03.14 einen Heiltrank für 5 Gold gekauft.

Was passierte zwischen tag 03 und tag 04, d.h. was änderte sich zwischen diesen Commits?

Benutzter Befehl:

`git log --oneline`

zeigt die Commit-Historie kurz und kompakt.

`git diff 2ffe6cf 39568d5`

vergleicht und zeigt was zwischen den beiden commits geändert wurde

Antwort:

```
PS C:\Users\Marco\Desktop\IntelliJ IDEA\Programmieren2\prog2_ybel_gitquest> git diff 2ffe6cf 39568d5
diff --git a/questlog.md b/questlog.md
index 70e2913..f1a7359 100644
--- a/questlog.md
+++ b/questlog.md
@@ -7,3 +7,5 @@
 Mit seinem Schwert in der Hand und einer leichten Rüstung bekleidet stieg M
 Immer wieder stieß Markus auf verschiedene Monster wie Goblins, Gnolle und sogar einen gefährlichen Schlammklumpen. Jeder Kampf war ein harter und erbitterter Kampf, bei dem Markus an seine Grenzen ging. Doch sein Wille, seine
 Quest zu erfüllen und das Amulet zu finden, gab ihm die Kraft, weiterzumachen.

 Schließlich erreichte Markus einen versteckten Raum im Dungeon, in dem ein mysteriöser alter Zwerg auf ihn wartete. Dieser Zwerg entpuppte sich als ein Händler und besaß einen kleinen Shop, in dem er nützliche Gegenstände und L
 ebensmittel verkaufte. Markus trat ein und begann mit dem Zwerg zu verhandeln.
+
+Erfrischt und gestärkt machte sich Markus auf den Weg, um die letzte Etappe seiner Quest zu erfüllen.
diff --git a/rucksack.md b/rucksack.md
index efc045a..86f99b2 100644
--- a/rucksack.md
+++ b/rucksack.md
@@ -2,7 +2,7 @@
 | Slot | Content |
 |-----|-----|
-| 0 | 10 Gold |
+| 0 | |
 | 1 | |
 | 2 | |
 | 3 | |
diff --git a/stats.md b/stats.md
index 0de3005..a655110 100644
--- a/stats.md
+++ b/stats.md
@@ -2,8 +2,8 @@
 | Property | Value |
 |-----|-----|
-| health | 5 |
+| health | 10 |
 | experience | 10 |
-| hunger | 10 |
+| hunger | 0 |
 | weapon | sword (3 dmg) |
 | armor | light (2 dmg) |
```

Hat der Held etwas gegessen? Falls ja, was und wann?

Benutzter Befehl:

`git log -p -- rucksack.md`

zeigt, wann rucksack.md geändert wurde und was genau in der Datei geändert wurde (von allen commits).

Antwort:

Der Held hat am Tag 03.17 ein Brot gegessen

2) Beim letzten Commit (tag 04.5) ist etwas schief gelaufen, es wurden versehentlich zu wenig `experience` Punkte eingestellt. Ändern Sie diesen letzten Commit und passen Sie die `experience` Punkte auf 42 an.

benutzte Befehle:

<code>notepad stats.md</code>	öffnet die Datei stats.md im Windows-Editor Notepad, damit man sie ansehen oder bearbeiten kann
<code>git status</code>	zeigt den aktuellen Zustand des Repositories (unter anderem welche Dateien geändert oder schon gestaged wurden)
<code>git add stats.md</code>	merkt die Datei stats.md für den nächsten Commit vor
<code>git commit --amend</code>	erstellt keinen neuen commit sondern überarbeitet den letzten

Problem 1:

Git hat den Texteditor Vim geöffnet, damit man die Commit-Nachricht für `git commit --amend` bearbeiten kann.

nach `git commit --amend` musste man noch

`Esc`
`:wq`
`Enter`

drücken um den amend-Commit abzuschließen.

Problem 2:

Nach `git log -p -- stats.md` war die Ansicht für die letzten stats falsch bzw. anders. Man musste (in IntelliJ)

`Alt + O` drücken
links auf `stats.md` draufgehen
die Änderung auswählen (links alt, rechts neu)
links auf Amend drücken
und dann zuletzt auf Amend Commit drücken

dannach hatte man wieder die normale Ansicht wie bei allen anderen stats auch.

3) Schreiben Sie die Geschichte in der Datei `questlog.md` fort und erzeugen Sie einen neuen Commit für `tag 04.6`. Ändern Sie bitte hierzu nur die eine Datei `questlog.md`.

benutzte Befehle:

<code>notepad questlog.md</code>	öffnet die Datei <code>questlog.md</code> im Windows-Editor Notepad, damit man sie ansehen oder bearbeiten kann
<code>git status</code>	zeigt den aktuellen Zustand des Repositories (unter anderem welche Dateien geändert oder schon gestaged wurden)
<code>git add questlog.md</code>	merkt die Datei <code>questlog.md</code> für den nächsten Commit vor
<code>git commit -m "tag 04.6"</code>	erstellt einen neuen commit mit einer Message

4) Schreiben Sie die Geschichte noch weiter fort (`tag 04.7`), aber ändern Sie diesmal mehrere Dateien, die an diesem Tag (neuer Commit) gemeinsam eingchecked werden sollen.

benutzte Befehle:

<code>notepad questlog.md</code>	öffnet die Datei <code>questlog.md</code> im Windows-Editor Notepad, damit man sie ansehen oder bearbeiten kann
<code>notepad stats.md</code>	öffnet die Datei <code>stats.md</code> im Windows-Editor Notepad, damit man sie ansehen oder bearbeiten kann
<code>git status</code>	zeigt den aktuellen Zustand des Repositories (unter anderem welche Dateien geändert oder schon gestaged wurden)
<code>git add .</code>	merkt alle geänderten oder neu hinzugefügte Dateien für den nächsten Commit vor
<code>git commit -m "tag 04.7"</code>	erstellt einen neuen commit mit einer Message

5) Fälschlicherweise wurden die Statuspunkte und die Ausrüstung bisher gemeinsam in der Datei `stats.md` geführt. Korrigieren Sie das und verschieben Sie die Ausrüstungsgegenstände aus der Datei `stats.md` in eine neue Datei `gear.md`. Checken Sie Ihre Änderungen als `tag 04.8` (neuer Commit) gemeinsam ein. (Hinweis: Es reicht, wenn diese Änderung als letzter Commit auf der Spitze des `master`-Branches existiert. Sie brauchen/sollen die Trennung von Statuspunkten und Ausrüstung nicht rückwirkend in die Historie einbauen!)

Erst habe ich eine neue Datei namens `gear.md` im gleichen Ordner wie die restlichen Dateien hinzugefügt und dort die beiden gears `weapon` und `armor` dort rein kopiert und eine Tabelle wie in `stats.md` erstellt. Die beiden neuen gears habe ich dann auch aus der Datei `stats.md` entfernt. (habe dann noch allgemein ein paar stats verändert und den Questlog weitergeführt)

benutzte Befehle:

<code>notepad questlog.md</code>	öffnet die Datei <code>questlog.md</code> im Windows-Editor Notepad, damit man sie ansehen oder bearbeiten kann
<code>notepad stats.md</code>	öffnet die Datei <code>stats.md</code> im Windows-Editor Notepad, damit man sie ansehen oder bearbeiten kann
<code>git status</code>	zeigt den aktuellen Zustand des Repositories (unter anderem welche Dateien geändert oder schon gestaged wurden)
<code>git add .</code>	merkt alle geänderten oder neu hinzugefügte Dateien für den nächsten Commit vor
<code>git commit -m "tag 04.8"</code>	erstellt einen neuen commit mit einer Message

Commit-Meldungen

Gute Commit-Meldungen schreiben erfordert Übung. Schauen Sie sich die beiden Commits [Dungeon-CampusMinden/Dungeon/commit/46530b6](#) und [Dungeon-CampusMinden/Dungeon/commit/3e37472](#) an.

Diskutieren Sie jeweils, was Ihnen an den Commits auffällt: Was gefällt Ihnen, was stört Sie? Schlagen Sie Verbesserungen vor.

Die Commit-Meldungen wurden in Stichpunkten sehr kurz und ungenau geschrieben, was jetzt tatsächlich geändert wurde. Diese sollten im besten Falle in ganzen Sätzen ausgeschrieben sein um zu verdeutlichen was genau verändert wurde.